

BACKEND REFERENCE

Kobie API Documentation

Complete reference for the Kobie loyalty-program intelligence backend: endpoint descriptions, request/response schemas, authentication, status codes, and end-to-end API flow — generated directly from the implemented FastAPI routes.

Base URL: `http://127.0.0.1:8000`

Framework: FastAPI (Uvicorn)

Endpoints documented: 13

Generated: 2026-07-02

Kobie API Documentation

Base URL: `http://127.0.0.1:8000` **Framework:** FastAPI (`fastapi>=0.115,<1`) served by Uvicorn (`server.sh` → `uvicorn` `server:app --host 127.0.0.1 --port 8000 --reload`) **Source of truth:** `server.py`

The Next.js frontend (`http://localhost:3000`) talks to this API through thin pass-through proxy routes under `frontend/app/api/**` ; those proxies carry no business logic of their own and are not documented separately.

Table of Contents

1. [Authentication](#)
 2. [Global Behavior](#)
 3. [Data Model Reference](#)
 4. [API Flow](#)
 5. [Endpoints](#)
 1. **Cache** — `GET /api/cache/check` , `GET /api/cache/check-multi`
 2. **Runs** — `POST /api/run` , `GET /api/run` , `GET /api/run/history` , `GET /api/run/{run_id}` , `DELETE /api/run/{run_id}` , `POST /api/run/{run_id}/delete` , `POST /api/run/{run_id}/stop`
 3. **Interaction** — `POST /api/run/{run_id}/clarify` , `POST /api/run/{run_id}/cache-decision` , `POST /api/run/{run_id}/converse` , `POST /api/run/{run_id}/compare/converse` , `POST /api/run/{run_id}/generate-brief`
 6. [Error Handling](#)
-

Authentication

None. There is no authentication or authorization layer on this API — no API keys, bearer tokens, sessions, or `Depends()` -based auth guards are present anywhere in `server.py` . Every endpoint is open to any client that can reach the host/port. The only access restriction is network-level: [CORS](#) limits which browser origins may call the API, and the server binds to `127.0.0.1` (loopback only) per `server.sh` .

If this API is ever exposed beyond localhost, an authentication layer must be added — this is not currently a defense against untrusted clients.

Global Behavior

CORS (`server.py:76-82`) — `CORSMiddleware` allows all methods and headers from:

- `http://localhost:3000`
- `http://127.0.0.1:3000`
- `http://localhost:3001`
- `http://127.0.0.1:3001`

Content type — All request bodies are JSON; all responses are JSON (`application/json`).

Concurrency model — `POST /api/run` starts the analysis pipeline on a background `threading.Thread` and returns immediately with a `run_id` . Clients poll `GET /api/run/{run_id}` to observe pipeline progress (`stage_status` , `active_stage` , `status`) until it reaches a terminal state (`done` , `error` , or `cancelled`).

Persistence — Runs are held in memory (`STORE` , keyed by `run_id`) while active, and persisted to a SQLite database (`core/db.py`) so history and cached program snapshots survive server restarts. `GET /api/run/{run_id}` and `GET /api/run/history` transparently fall back to the DB when a run is no longer in memory.

Shutdown — On process shutdown, the SQLite DB is checkpointed (`server.py:69-74`).

Data Model Reference

Run status values (`status` field)

VALUE	MEANING
<code>running</code>	Pipeline is actively executing.
<code>clarification_needed</code>	Input validator needs user disambiguation — call <code>POST /api/run/{run_id}/clarify</code> .
<code>cache_hit_pending</code>	A cached analysis was found for the program; awaiting user choice — call <code>POST /api/run/{run_id}/cache-decision</code> .
<code>done</code>	Pipeline completed successfully (all programs succeeded, for compare mode: <code>any_success</code> / <code>success</code> was true).
<code>error</code>	Pipeline failed.
<code>cancelled</code>	Run was stopped via <code>POST /api/run/{run_id}/stop</code> , or is an orphaned "running" DB row from a server restart.

Pipeline stage IDs (`stage_status` keys, `UI_STAGES`)

`input_validator` , `query_generator` , `retrieval` , `firecrawl_scraper` , `chunking` , `extraction` , `claims` , `adjudication` , `output` — each maps to `"idle"` | `"running"` | `"done"` | `"error"` .

Run modes (`mode`)

VALUE	DESCRIPTION
<code>single</code>	Analyze one loyalty program.
<code>compare</code>	Analyze and compare two or more programs.
<code>converse</code>	(Accepted by the API; behaves like a normal run — conversation happens post-hoc via the converse endpoints regardless of mode.)

Any `mode` value other than `single` , `compare` , or `converse` sent to `POST /api/run` silently falls back to `single` .

Core object shapes

These Pydantic models (defined in `core/schemas.py`) appear embedded in run responses:

ClaimStatus (enum): `supported` · `conflicting` · `not_found/manual_review_needed` · `null` · `rejected_unsupported`

FieldReportEntry

```
{
  "field_path": "earn_mechanics.base_earn_rate",
  "category": "earn_mechanics",
  "status": "extracted",
  "value": "5 miles per $1 on Delta flights",
  "source_urls": ["https://..."],
  "source_snippet": "string | null",
  "confidence": 0.93,
  "corroboration_count": 3,
  "rejected_alternatives": [{"value": "...", "source_urls": ["..."], "reason": "..."}],
  "all_values": null,
  "conflict_type": null
}
```

FieldReport

```
{
  "entity_name": "Delta SkyMiles",
  "generated_at": "2026-07-02T12:00:00+00:00",
  "entries": [ /* FieldReportEntry[] */ ],
  "extracted_count": 40,
  "ambiguous_count": 3,
  "not_found_count": 5,
  "flagged_count": 0
}
```

BriefOutput (single-program narrative brief)

```
{
  "brief_id": "brief...",
  "run_id": "run...",
  "brief_text": "string",
  "cited_claim_ids": ["claim..."],
  "word_count": 512,
  "entailment_passed": true,
  "unsupported_sentences": []
}
```

ComparisonBrief (compare-mode narrative brief)

```
{
  "brief_id": "compbrief_...",
  "run_id": "run_...",
  "programs": ["Delta SkyMiles", "Marriott Bonvoy"],
  "overall_winner": "Delta SkyMiles",
  "executive_summary": "string",
  "category_verdicts": [
    {
      "category": "earn_mechanics",
      "label": "string",
      "winner": "Delta SkyMiles",
      "insight": "string",
      "source_urls": []
    }
  ],
  "key_differentiators": [
    {
      "topic": "string",
      "insight": "string",
      "advantage": "Delta SkyMiles",
      "source_urls": [],
      "rejected_note": null
    }
  ],
  "personas": [
    { "program": "Delta SkyMiles", "best_for": "string" }
  ],
  "strategic_profiles": [
    { "program": "Delta SkyMiles", "advantages": [], "gaps": [] }
  ],
  "differentiation_themes": [
    { "theme": "string", "summary": "string", "leader": "Delta SkyMiles" }
  ],
  "generated_at": "2026-07-02T12:00:00+00:00"
}
```

ConverseAnswer (returned by both converse endpoints)

```
{
  "answer": "string",
  "status": "supported",
  "cited_claim_ids": ["claim_..."],
  "missing_field_paths": [],
  "source_urls": ["https://..."]
}
```

API Flow

Single-program analysis

1. `GET /api/cache/check?q=<program>` — optionally check for a cached analysis before starting.
2. `POST /api/run` — start the pipeline (`mode: "single"`); receive `run_id` .
3. Poll `GET /api/run/{run_id}` until `status` is `done` , `error` , or `cancelled` .
 - If `status` becomes `clarification_needed` , call `POST /api/run/{run_id}/clarify` with the user's answer, then resume polling.
 - If `status` becomes `cache_hit_pending` , call `POST /api/run/{run_id}/cache-decision` with `"use_cache"` or `"fresh"` , then resume polling.
4. Once `done` , optionally call `POST /api/run/{run_id}/converse` repeatedly to ask grounded questions about the `final_brief` .
5. `DELETE /api/run/{run_id}` (or `POST /api/run/{run_id}/delete`) to remove the run, or `POST /api/run/{run_id}/stop` to cancel an in-progress run.

Compare analysis

1. `GET /api/cache/check-multi?programs=A&programs=B` — optionally batch-check cache for all programs.
2. `POST /api/run` with `mode: "compare"` and `programs: ["A", "B", ...]` (or `user_input` + `user_input_b` for exactly two).
3. Poll `GET /api/run/{run_id}`; `comparison_run` in the response tracks `current_program_index`, `program_statuses`, and `program_stage_statuses` as each program is analyzed sequentially.
4. Once complete, `comparison_brief` is populated (or call `POST /api/run/{run_id}/generate-brief` to generate/backfill it for a stored run).
5. Call `POST /api/run/{run_id}/compare/converse` for grounded Q&A over the comparison.

History / cross-session

- `GET /api/run/history` lists all persisted + live runs (works after a server restart, since it reads the DB).

Endpoints

Cache Endpoints

`GET /api/cache/check`

Check whether a single program already has a stored analysis in the DB cache.

- **Auth:** none
- **File:** `server.py:1729`

Query Parameters

NAME	TYPE	REQUIRED	DESCRIPTION
<code>q</code>	string	yes	Program name to look up.

Example Request

```
GET /api/cache/check?q=Delta%20SkyMiles
```

Response `200 OK` — cache miss:

```
{ "found": false }
```

Response `200 OK` — cache hit:

```
{
  "found": true,
  "program_name": "Delta SkyMiles",
  "brand": "Delta Air Lines",
  "country_or_region": "US",
  "run_date": "2026-06-30",
  "run_datetime": "2026-06-30 14:02 UTC",
  "run_timestamp": "2026-06-30T14:02:11+00:00",
  "age_days": 2
}
```

Status Codes: `200` only — no error paths.

`GET /api/cache/check-multi`

Batch cache check for compare mode — one result per program, in the order requested.

- **Auth:** none
- **File:** `server.py:1738`

Query Parameters

NAME	TYPE	REQUIRED	DESCRIPTION
<code>programs</code>	string (repeated)	no (defaults to <code>[]</code>)	Repeat the parameter once per program, e.g. <code>?programs=Delta&programs=United</code> .

Example Request

```
GET /api/cache/check-multi?programs=Delta%20SkyMiles&programs=United%20MileagePlus
```

Response `200 OK`

```
[
  {
    "found": true,
    "program_name": "Delta SkyMiles",
    "brand": "Delta Air Lines",
    "country_or_region": "US",
    "run_date": "2026-06-30",
    "run_datetime": "2026-06-30 14:02 UTC",
    "run_timestamp": "2026-06-30T14:02:11+00:00",
    "age_days": 2,
    "program": "Delta SkyMiles"
  },
  { "found": false, "program": "United MileagePlus" }
]
```

Status Codes: `200` only.

Run Endpoints

`POST /api/run`

Start a new analysis run. The pipeline executes asynchronously on a background thread; this call returns immediately with a `run_id` to poll.

- **Auth:** none
- **Status Code (success):** `201 Created`
- **File:** `server.py:1824`

Request Body (`CreateRunBody`)

FIELD	TYPE	DEFAULT	DESCRIPTION
<code>user_input</code>	string	— (required, non-blank)	Program name / free-text query. In compare mode with only two programs, this is program A.
<code>mode</code>	string	<code>"single"</code>	<code>"single"</code> , <code>"compare"</code> , or <code>"converse"</code> . Any other value silently falls back to <code>"single"</code> .
<code>user_input_b</code>	string null	<code>null</code>	Program B, for a 2-program compare run (alternative to <code>programs</code>).
<code>programs</code>	string[] null	<code>null</code>	Full list of programs for compare mode (used when comparing more than 2, or instead of <code>user_input_b</code>). Requires ≥ 2 non-blank entries to take effect.
<code>mock</code>	boolean	<code>false</code>	If true, skips live scraping/retrieval and runs against pre-built mock field data (for demos/testing).
<code>force_fresh</code>	boolean	<code>false</code>	If true, bypasses the cache-hit pause and always performs a fresh analysis.

Example Request

```
POST /api/run
Content-Type: application/json

{
  "user_input": "Delta SkyMiles",
  "mode": "single"
}
```

Example Request — compare mode

```
POST /api/run
Content-Type: application/json

{
  "user_input": "Delta SkyMiles",
  "mode": "compare",
  "programs": ["Delta SkyMiles", "Marriott Bonvoy"]
}
```

Response `201 Created`

```
{ "run_id": "run_3f9a1c2b8e4d4f0aa1c2b8e4d4f0aa1c" }
```

Error Responses

STATUS	CONDITION	BODY
<code>400 Bad Request</code>	<code>user_input</code> is empty or whitespace-only	<code>{"detail": "user_input is required"}</code>

`GET /api/run`

List summaries of all runs currently held in memory (does not include runs only present in the DB — use `GET /api/run/history` for that).

- **Auth:** none
- **File:** `server.py:1850`

Response `200 OK`

```
[
  {
    "run_id": "run_3f9a1c2b...",
    "user_input": "Delta SkyMiles",
    "mode": "single",
    "data_quality": 0.87,
    "status": "done",
    "created_at": "2026-07-02T10:15:00+00:00"
  }
]
```

Sorted by `created_at` descending.

Status Codes: `200` only.

`GET /api/run/history`

Return persisted analyses (from the DB) merged with any live in-memory runs — this is the endpoint that survives server restarts.

- **Auth:** none
- **File:** `server.py:1753`

Response `200 OK`

```
[
  {
    "run_id": "run_3f9a1c2b...",
    "user_input": "Delta SkyMiles",
    "mode": "single",
    "program_name": "Delta SkyMiles",
    "data_quality": 0.87,
    "status": "done",
    "created_at": "2026-07-02T10:15:00+00:00",
    "source": "db"
  }
]
```

Notes:

- `source` is `"db"` for persisted rows or `"live"` for in-memory runs.
- `program_name` is `"A vs B"` for compare runs.
- A DB row with `status: "running"` that has no corresponding live run (e.g. after a server restart mid-run) is relabeled `status: "cancelled"` in this response.
- Sorted by `created_at` descending.

Status Codes: `200` only.

`GET /api/run/{run_id}`

Poll the full state of a single run — pipeline field data, `stage_status`, `status`, `conversation`, cost report, and (for compare mode) per-program progress.

- **Auth:** none
- **File:** `server.py:1861`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	The run identifier returned by <code>POST /api/run</code> .

Response `200 OK` (abridged — full shape mirrors `AgentState` plus run metadata; see [Data Model Reference](#))

```
{
  "run_id": "run_3f9a1c2b...",
  "mode": "single",
  "user_input": "Delta SkyMiles",
  "program_name": "Delta SkyMiles",
  "brand": "Delta Air Lines",
  "domain": "delta.com",
  "field_report": { "...": "FieldReport" },
  "final_brief": { "...": "BriefOutput" },
  "data_quality": 0.87,
  "errors": [],
  "stage_status": {
    "input_validator": "done",
    "query_generator": "done",
    "retrieval": "done",
    "firecrawl_scraper": "done",
    "chunking": "done",
    "extraction": "done",
    "claims": "done",
    "adjudication": "done",
    "output": "done"
  },
  "active_stage": null,
  "status": "done",
  "conversation": [],
  "cost_report": { "...": "per-stage LLM cost breakdown" },
  "run_started_at": "2026-07-02T10:15:00+00:00",
  "run_finished_at": "2026-07-02T10:17:42+00:00",
  "comparison_conversation": [],
  "created_at": "2026-07-02T10:15:00+00:00",
  "updated_at": "2026-07-02T10:17:42+00:00"
}
```

Additional fields present only mid-run:

- `cache_hit` — present when `status` is `cache_hit_pending`: `{"program_name", "brand", "run_date", "age_days"}`.

Additional fields present only in compare mode (`mode: "compare"`):

```
{
  "compare_b": { "...": "second program's serialized identity, if resolved" },
  "comparison_run": {
    "programs": ["Delta SkyMiles", "Marriott Bonvoy"],
    "current_program_index": 1,
    "total_programs": 2,
    "program_statuses": ["done", "running"],
    "program_states": [ "...", null ],
    "program_stage_statuses": [ "...", "..." ]
  }
}
```

Fallback behavior: if the run is no longer in memory, the server reconstructs the response from the DB `runs` table, then `run_snapshots`, then (if the run row exists but has no persisted state — e.g. server restarted mid-run) returns a synthetic response with `status: "cancelled"`.

Error Responses

STATUS	CONDITION	BODY
404 Not Found	<code>run_id</code> does not exist in memory, the <code>runs</code> table, or <code>run_snapshots</code>	<code>{"detail": "run not found"}</code>

DELETE /api/run/{run_id}

Remove a run from the in-memory store and the persistent DB (including any associated program cache snapshots).

- **Auth:** none
- **File:** `server.py:1904`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	The run identifier to delete.

Response **200 OK**

```
{ "ok": true, "deleted": true }
```

`deleted` reflects whether a DB row existed and was removed; this call is idempotent and returns **200** even if the run did not exist.

Status Codes: **200** only.

POST /api/run/{run_id}/delete

Functionally identical alias for `DELETE /api/run/{run_id}` — added because the Next.js proxy layer had trouble forwarding `DELETE` requests.

- **Auth:** none
- **File:** `server.py:1909`

Same path parameters, response body, and status codes as `DELETE /api/run/{run_id}` above.

POST /api/run/{run_id}/stop

Cancel an in-progress run (running, awaiting clarification, or awaiting a cache decision).

- **Auth:** none
- **File:** `server.py:2102`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	The run identifier to stop.

Response **200 OK**

```
{ "ok": true }
```

Sets `status` to `cancelled` and signals the background pipeline thread to exit (also unblocks any pending clarification/cache-decision wait so the thread doesn't hang).

Error Responses

STATUS	CONDITION	BODY
404 Not Found	<code>run_id</code> not found in memory	<code>{"detail": "run not found"}</code>
400 Bad Request	Run's <code>status</code> is not one of <code>running</code> , <code>clarification_needed</code> , <code>cache_hit_pending</code>	<code>{"detail": "run is not in progress"}</code>

Interaction Endpoints

POST `/api/run/{run_id}/clarify`

Submit a clarification answer when the input validator stage needs disambiguation (`status: "clarification_needed"`).

- **Auth:** none
- **File:** `server.py:1915`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	The run identifier awaiting clarification.

Request Body (`ClarifyRequest`)

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>answer</code>	string	yes (non-blank)	The user's disambiguation answer (e.g. picking among multiple matched programs, or a corrected program name).

Example Request

```
POST /api/run/run_3f9a1c2b.../clarify
Content-Type: application/json

{ "answer": "The airline one, Delta Air Lines" }
```

Response **200 OK**

```
{ "ok": true }
```

Unblocks the waiting pipeline thread, which resumes the `input_validator` stage with the new answer appended to `validation_messages`.

Error Responses

STATUS	CONDITION	BODY
404 Not Found	<code>run_id</code> not found in memory	<code>{"detail": "run not found"}</code>
400 Bad Request	<code>answer</code> is empty/whitespace	<code>{"detail": "answer is required"}</code>
400 Bad Request	Run's <code>status</code> is not <code>clarification_needed</code>	<code>{"detail": "run is not waiting for clarification"}</code>

POST /api/run/{run_id}/cache-decision

Resolve a pending cache-hit pause (`status: "cache_hit_pending"`) — choose to reuse the cached analysis or force a fresh run.

- **Auth:** none
- **File:** `server.py:1957`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	The run identifier awaiting a cache decision.

Request Body (`CacheDecisionRequest`)

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>decision</code>	string	yes	Must be <code>"use_cache"</code> or <code>"fresh"</code> .

Example Request

```
POST /api/run/run_3f9a1c2b.../cache-decision
Content-Type: application/json

{ "decision": "use_cache" }
```

Response 200 OK

```
{ "ok": true }
```

Error Responses

STATUS	CONDITION	BODY
404 Not Found	<code>run_id</code> not found in memory	<code>{"detail": "run not found"}</code>
400 Bad Request	<code>decision</code> is not <code>"use_cache"</code> or <code>"fresh"</code>	<code>{"detail": "decision must be 'use_cache' or 'fresh'"}</code>
400 Bad Request	Run's <code>status</code> is not <code>cache_hit_pending</code>	<code>{"detail": "run is not waiting for a cache decision"}</code>

POST /api/run/{run_id}/converse

Ask a grounded question about a completed single-program run's brief and field report.

- **Auth:** none

- **File:** `server.py:1973`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	A completed (or historical, DB-persisted) single-mode run.

Request Body (`ConverseRequest`)

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>message</code>	string	yes (non-blank)	The user's question about the analyzed program.

Example Request

```
POST /api/run/run_3f9a1c2b.../converse
Content-Type: application/json

{ "message": "What is the base earn rate?" }
```

Response **200 OK** (`ConverseAnswer`)

```
{
  "answer": "Delta SkyMiles members earn 5 miles per $1 spent on Delta-marketed flights.",
  "status": "supported",
  "cited_claim_ids": ["claim_1a2b3c"],
  "missing_field_paths": [],
  "source_urls": ["https://www.delta.com/skymiles"]
}
```

Behavior: if the run is live in memory, answers from the in-memory `final_brief` / `field_report` ; otherwise falls back to loading the persisted brief from the DB `runs` or `run_snapshots` tables. Each question/answer pair is appended to the run's `conversation` history (visible in `GET /api/run/{run_id}`).

Error Responses

STATUS	CONDITION	BODY
400 Bad Request	<code>message</code> is empty/whitespace	<code>{"detail": "message is required"}</code>
400 Bad Request	Live run has no <code>final_brief</code> yet	<code>{"detail": "Pipeline has not completed – no brief available yet."}</code>
404 Not Found	Run not found in memory or DB (DB fallback path)	<code>{"detail": "run not found"}</code>
400 Bad Request	No persisted brief available, or brief fails to parse (DB fallback path)	<code>{"detail": "<context-specific message>"}</code>
500 Internal Server Error	The underlying <code>answer_question</code> call raises an exception	<code>{"detail": "<exception message>"}</code>

POST `/api/run/{run_id}/compare/converse`

Ask a grounded question about a completed compare-mode run's comparison brief.

- **Auth:** none

- File: `server.py:1095`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	A completed (or historical) compare-mode run.

Request Body (`ConverseRequest`)

FIELD	TYPE	REQUIRED	DESCRIPTION
<code>message</code>	string	yes (non-blank)	The user's question comparing the analyzed programs.

Example Request

```
POST /api/run/run_3f9a1c2b.../compare/converse
Content-Type: application/json

{ "message": "Which program has better redemption value?" }
```

Response **200 OK** (`ConverseAnswer` — same shape as the single-program converse endpoint)

```
{
  "answer": "Marriott Bonvoy generally offers better redemption value at the lower award categories, while Delta SkyMiles is more competitive for premium cabin redemptions.",
  "status": "supported",
  "cited_claim_ids": ["claim_9f8e7d"],
  "missing_field_paths": [],
  "source_urls": ["https://www.marriott.com/loyalty.mi"]
}
```

Behavior: analogous to the single-program converse endpoint, but resolves against `comparison_brief` and each program's `FieldReport`. Falls back to the DB for historical runs and can on-demand-generate a comparison brief if only a stub exists. Each pair is appended to `comparison_conversation`.

Error Responses

STATUS	CONDITION	BODY
400 Bad Request	<code>message</code> is empty/whitespace	<code>{"detail": "message is required"}</code>
400 Bad Request	Run's <code>mode</code> is not <code>compare</code>	<code>{"detail": "not a comparison run"}</code>
400 Bad Request	No comparison brief yet on a live run	<code>{"detail": "Comparison brief not yet available – please wait for the run to complete."}</code>
404 Not Found	Run not found in memory or DB (DB fallback path)	<code>{"detail": "run not found"}</code>
400 Bad Request	Run state not persisted, no brief available, or brief fails to parse (DB fallback path)	<code>{"detail": "<context-specific message>"}</code>
500 Internal Server Error	The underlying <code>answer_comparison_question</code> call raises an exception	<code>{"detail": "<exception message>"}</code>

POST `/api/run/{run_id}/generate-brief`

Generate (or return the already-cached) comparison brief for a stored compare run. Used to backfill a brief for historical runs that only have a placeholder/stub brief.

- **Auth:** none
- **File:** `server.py:2013`

Path Parameters

NAME	TYPE	DESCRIPTION
<code>run_id</code>	string	A compare-mode run (live or historical).

Request Body: none

Example Request

```
POST /api/run/run_3f9a1c2b.../generate-brief
```

Response `200 OK` (`ComparisonBrief` — see [Data Model Reference](#))

```
{
  "brief_id": "compbrief_...",
  "run_id": "run_3f9a1c2b...",
  "programs": ["Delta SkyMiles", "Marriott Bonvoy"],
  "overall_winner": "Delta SkyMiles",
  "executive_summary": "...",
  "category_verdicts": [ "..." ],
  "key_differentiators": [ "..." ],
  "personas": [ "..." ],
  "strategic_profiles": [ "..." ],
  "differentiation_themes": [ "..." ],
  "generated_at": "2026-07-02T10:17:42+00:00"
}
```

Behavior: returns the existing brief immediately if one with populated `category_verdicts` already exists (in memory or DB). Otherwise rebuilds `FieldReport` s from each program's stored state, calls the brief generator, and persists the result back to the DB `runs` table so subsequent calls are served from cache.

Error Responses

STATUS	CONDITION	BODY
<code>404 Not Found</code>	<code>run_id</code> not found in DB	<code>{"detail": "run not found"}</code>
<code>400 Bad Request</code>	Run's <code>mode</code> is not <code>compare</code>	<code>{"detail": "not a comparison run"}</code>
<code>400 Bad Request</code>	Run state not available in DB	<code>{"detail": "run state not available"}</code>
<code>400 Bad Request</code>	Fewer than 2 programs have usable field-report data	<code>{"detail": "Not enough program field data to generate brief. Re-run the comparison."}</code>
<code>500 Internal Server Error</code>	Brief generation raises an exception	<code>{"detail": "Brief generation failed: <exception>"}</code>

Error Handling

All errors follow FastAPI's standard `HTTPException` shape:

```
{ "detail": "human-readable error message" }
```


STATUS	MEANING IN THIS API
400 Bad Request	Invalid or missing input, or the run is not in the required state for the requested action.
404 Not Found	<code>run_id</code> does not exist in memory, the <code>runs</code> table, or <code>run_snapshots</code> .
500 Internal Server Error	An unhandled exception occurred inside a converse/brief-generation call; <code>detail</code> contains the raw exception string.

There is no standardized machine-readable error code — `detail` is a free-text message intended for surfacing directly in the UI.